Natural Language to RPI Selection Rules POC

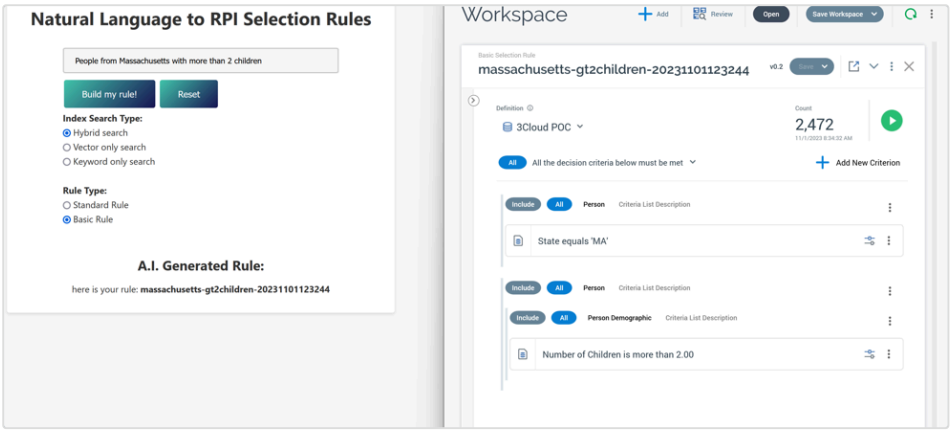
Document summary

This document covers the technical aspects of an R&D proof of concept project that demonstrates techniques for translating natural language descriptions of segments into RPI selection rules.

Technical overview

Project files can be found in the ResearchProject repo in the folder 'NLP to RPI POC'

The solution is a basic python web app that takes a user input describing a marketing segment (e.g. People from Boston who spent more than $200 last month) and then builds an RPI selection rule equivalent of the natural language description of the segment by using the Azure OpenAI ChatGPT-4 model to construct valid RPI Integration API payloads and then issuing the calls to the API. Here is a screenshot of the app and the resulting rule:



There is an option to control how to do the initial index search (for selecting which attributes to pass to the model - more on that later!) and another option specifying whether to create a standard or basic selection rule.

The core approach to this solution leverages prompt engineering techniques combined with the popular RAG (Retrieval Augmented Generation) method where the prompt is enhanced and grounded with data that the model may not have been trained on (in our case the id's, names, and sample values of RPI attributes).

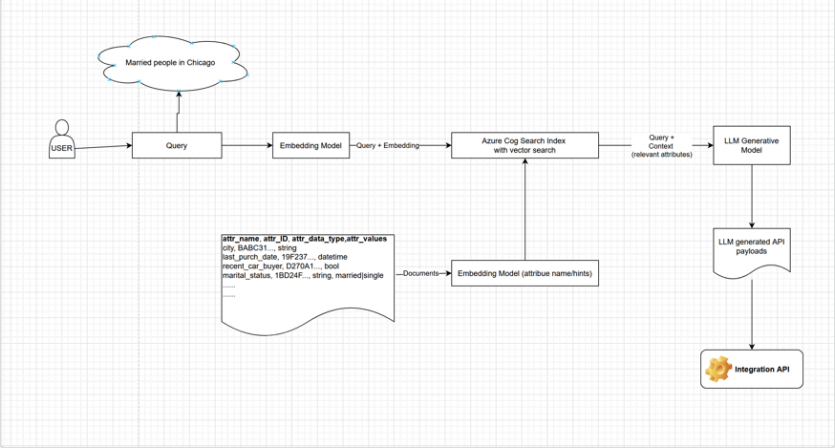
Here are a few good documents that helped guide some of the work for this POC; the first on prompt engineering techniques and the other 2 covering RAG methodology and pros/cons.

System message design for Azure OpenAI - Microsoft Foundry

Retrieval Augmented Generation: Grounding AI Responses in Factual Data

Build a RAG agent with LangChain - Docs by LangChain

In our solution, we've taken the approach of creating an Azure cognitive search index containing information about RPI attributes and creating vectors for some of the index field to support a hybrid vector + keyword search and leveraging this index to help inform the model on which attributes can be used in constructing the payloads for selection rule creation. Here is a simplified diagram of the flow:



Notes on dependencies and environment variables

RPI dependencies

The POC uses the 'Transcend19' tenant in the RPI Lab instance (rpilab.rphelios.net). For this solution to work, the tenant must have a standard resolution level setup, as well as a basic SQL definition configured. Also, the IntegrationAPI is required as this is what is used to create the rules. At the time of writing the version of RPI used for the POC was v6.6.23233

Azure dependencies

- **Azure Open AI** - the solution requires Azure Open AI resources with gpt-4-8K (or gpt-4-32K) and text-embedding-ada-002 models deployed. In our project, we used the rp-eng-openai-eus2 resource in the RP-Engineering subscription
- **Azure Cognitive Services** - we also require an Azure Cognitive Search service resource in which we create the indexes used for attribute search. In our project, we used the cogsearch-eng-openai resource in the RP-Engineering subscription
- **Blob Storage** - this is required to store the files used to construct the index

Environment variables

Within the project there is a `.env.sample` file that contains all of the environment variables that need to be set for the project work. This needs to be filled according the the environment and then renamed to `.env` before starting the project. A lot of the variables are self-explanatory but here are the descriptions. The most up-to-date description can be found in the project's README.md file.

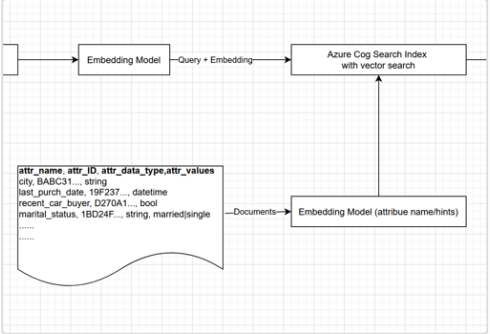
rpi_oauth_client_id - the OAuth client ID used by the Integration API. Normally would be set to 'rpiwebapi'
rpi_client_id - the RPI tenant's client ID (used to make Integration API calls)
rpi_folder_id - the RPI folder ID where standard RPI Selection rules should be created
rpi_resolution_id - the resolution ID to use when creating standard rules
rpi_folder_id_basic_rules - the RPI folder ID where basic rules should be created
rpi_sql_definition_id - the SQL definition ID used when creating basic RPI selection rules
oauth_client_secret - the oauth client secret used by the Integration API
oauth_user - the RPI user to be used for authentication
oauth_pw - the RPI password to be used for authentication
oauth_token_url - the RPI Integration API token url *(e.g. https://<host:port>/IntegrationAPI/token)*
rpi_api_base_url - the RPI Integration API base (e.g. https://<host:port>/IntegrationAPI/api/v1/)
OPENAI_API_KEY - Azure Open AI service API key
OPENAI_API_TYPE - should be set to 'azure' but theoretically this could work with openAI as well
OPENAI_API_BASE - endpoint for Azure Open AI service *(e.g. https://<servicename>.openai.azure.com/)*
OPENAI_API_VERSION - OpenAI API version, in our case set to '2023-07-01-preview'
OPENAI_CHATGPT_TEMP - ChatGPT temperature setting, in our project this is set to '0.7'
OPENAI_CHATGPT_ENGINE - name of the chatGPT deployment. In our case this is set to 'gpt-4-32k' (or 8K)
AZURE_SEARCH_ENDPOINT - Azure cognitive search URL *(e.g. https//<servicename>.search.windows.net)*
AZURE_SEARCH_KEY - Azure cognitive search API key

AZURE_SEARCH_INDEX_STANDARD_RULE - the name of the index to use for standard rules
AZURE_SEARCH_INDEX_BASIC_RULE - the name of the index to use for basic rules

Solution components

Azure index with vector search

A key component of this solution is the vector-enabled index containing information about the RPI attributes. In the flow from above, we're talking about everything related to this section:



There is an excellent resource/reference repo available from Microsoft that covers all the essential information a developer would need for understanding this service and how to use it. Highly recommend reviewing the examples here: GitHub - Azure/azure-search-vector-samples: A repository of code samples for Vector search capabilities in Azure AI Search.

Preparing the index file

In our project, there is a folder called 'embeddings' that contains some helpful python scripts we used to simplify creating the source file for the index:

gen_basic_sql_def_json.py

This script:

- queries the Integration API endpoint for the SQL database definition (***https://<host:port>/IntegrationAPI/api/v1/client/configuration/sql-database-definition/by-name?message.name=<SQLDefName>***)
- recurses through the file and builds out a list of all the attributes and for any string attribute, it attempts to retrieve the cached values (***https://<host:port>/IntegrationAPI/api/v1/client/attribute-values?message.id=<attributeID>***)

> ⚠ The attribute information returned from the Integration API doesn't return any information that would allow us to distinguish a Boolean from an Integer - both types of attributes are returned as Integers so had to manually update the datatype to Boolean for the boolean attribute TODO: log bug with Jim

- The output is a JSON file with records that look like this:


```
[
    {
        "attribute_name": "City",
        "attribute_group_id": "1589600d-6936-4968-9ed0-f70902a759b5",
        "attribute_datatype": "String",
        "attribute_id": "51910595-9166-4a2e-bc13-103bee58caa8",
        "attribute_sample_values": "NEW YORK | HOUSTON | CHICAGO | LOS ANGELES | CLEVELAND | DALLAS | MIAMI | BROOKLYN | SAN FRANCISCO |
WASHINGTON | SAN ANTONIO | PHOENIX | PHILADELPHIA | PORTLAND | SAN DIEGO | SEATTLE | ATLANTA | AUSTIN | CINCINNATI | COLUMBUS | LAS
VEGAS | BALTIMORE | INDIANAPOLIS | SPRINGFIELD | RICHMOND | ST LOUIS | DENVER | FLUSHING | SAN JOSE | PITTSBURGH | TAMPA | NASHVILLE |
KANSAS CITY | OKLAHOMA CITY | SACRAMENTO | COLUMBIA | JACKSONVILLE | MINNEAPOLIS | GREENVILLE | TUCSON | ORLANDO | DETROIT | MEMPHIS |
JACKSON | ALBUQUERQUE | MILWAUKEE | BOSTON | FORT LAUDERDALE | ROCHESTER | FORT WORTH | WILMINGTON | CHARLOTTE | BIRMINGHAM | TULSA |
LOUISVILLE | OMAHA | DAYTON | BRONX | GRAND RAPIDS | NEW ORLEANS | KNOXVILLE | EL PASO | COLORADO SPRINGS | MADISON | LEXINGTON | SALT
LAKE CITY | RALEIGH | ARLINGTON | CHARLESTON | HONOLULU | GREENSBORO | SALEM | OAKLAND | WEST PALM BEACH | CANTON | FRESNO | ALBANY |
SPOKANE | BAKERSFIELD | NEWARK | LITTLE ROCK | LONG BEACH | MARIETTA | ST PAUL | BATON ROUGE | JAMAICA | ANCHORAGE | PASADENA | TROY |
BURLINGTON | ORANGE | LEBANON | LAFAYETTE | SYRACUSE | BUFFALO | ALEXANDRIA | WICHITA | LINCOLN | FAYETTEVILLE | LANCASTER | ANAHEIM |
DECATUR | MESA | TALLAHASSEE | AUBURN | STOCKTON | NAPLES | FAIRFIELD | SARASOTA | MONROE | TACOMA | BLOOMINGTON | GLENDALE | HOLLYWOOD
| BOISE | MARION | TRENTON | AURORA | AKRON | SCOTTSDALE | FORT MYERS | PLYMOUTH | LUBBOCK | CORPUS CHRISTI | ATHENS | CONCORD | POMPANO
BEACH | GAINESVILLE | HARRISBURG | AMARILLO | BOULDER | STATEN ISLAND | TOLEDO | PENSACOLA | WARREN | VIRGINIA BEACH | SANTA ANA |
CAMBRIDGE | HENDERSON | FORT WAYNE | HUNTSVILLE | ANN ARBOR | EUGENE | FREMONT | ENGLEWOOD | DURHAM | FRANKLIN | ST PETERSBURG | RENO |
MOBILE | FLORENCE | DES MOINES | ROCKFORD | WORCESTER | LANSING | PROVIDENCE | FLINT | TEMPE | FORT COLLINS | PEORIA | CLINTON | SANTA
ROSA | METAIRIE | ROANOKE | MIDDLETOWN | CLEARWATER | SHAWNEE MISSION | MIDLAND | ASHLAND | EVANSVILLE | RIVERSIDE | TORRANCE | HIALEAH
| SANTA BARBARA | NEWPORT | BEAUMONT | MANCHESTER | SHREVEPORT | MACON | BELLEVILLE | BETHESDA | TYLER | PRINCETON | CHESAPEAKE |
```

```
    FAIRFAX | BOCA RATON | NORFOLK | MODESTO | MT VERNON | SANTA FE | AUGUSTA | SAVANNAH | LAWRENCEVILLE | DULUTH | WALNUT CREEK | JERSEY
    CITY | NORWALK | CHATTANOOGA | READING | NEWTON | None"
    },
    {
        "attribute_name": "Country",
        "attribute_group_id": "1589600d-6936-4968-9ed0-f70902a759b5",
        "attribute_datatype": "String",
        "attribute_id": "e14f5437-f2f0-4437-83be-043a5ad1ac34",
        "attribute_sample_values": "United States |  | None"
    },
    .....
```

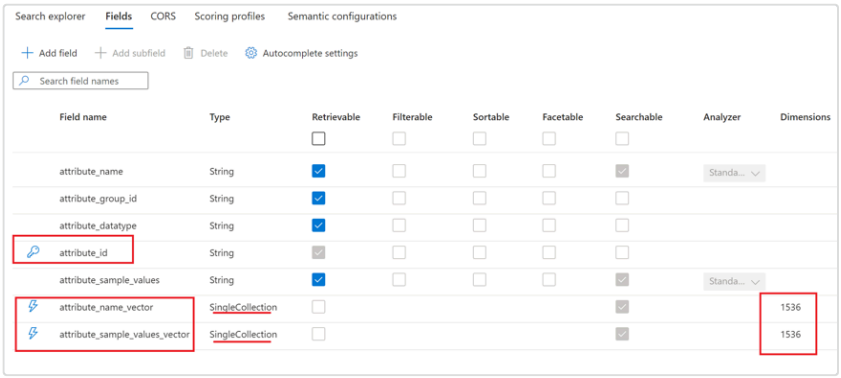
create_embeddings_basic_rule_attribs.py

This script uses the `text-embedding-ada-002` model to generate embeddings for the attribute_name and attribute_sample_values fields and creates a new files containing the extra vector fields. The final output of this method is what should be uploaded to blob storage when creating the index.

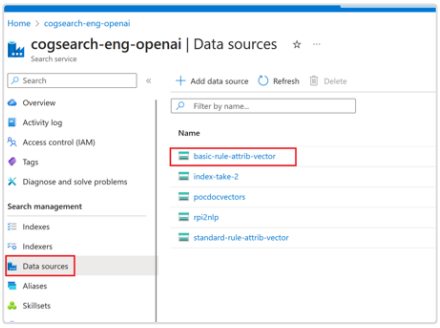
Creating the index in Azure

To manually create the index, we essentially need to defined the index, create a data source, and run and indexer job. This is detailed here:

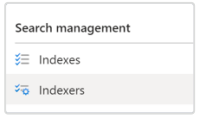
- From the Azure search service portal - create a new index and construct exactly as shown below, making sure that (1) 'attribute_id' is set to be the key, (2) 'attribute_name_vector' and 'attribute_sample_values_vector' are set to be type SingleCollection, and (3) Dimensions for those 2 vector fields is set to 1536. Other than that, keep the defaults when setting up the vector configuration

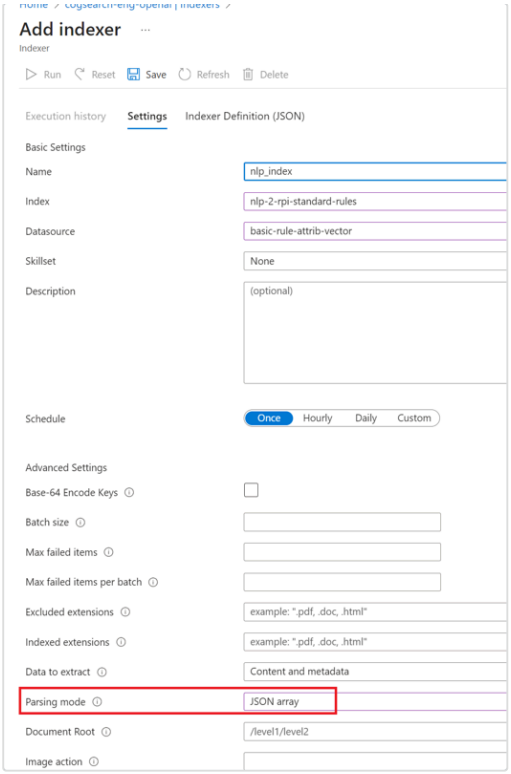


- From the Azure search service portal - create a new data source pointing to a blob storage folder containing the index file created above (the file with the 2 vector fields)



- from the Azure search service portal - create a new indexer job, pointing to the index and data source created above, and **make sure to set the parsing mode to JSON Array. ←important!**





- The indexer should run in a few seconds and you can check the run history to confirm a successful run:

The index is now ready to be used! The name of the index should be set in the .env variable: `AZURE_SEARCH_INDEX_BASIC_RULE`

TODO: provide example of automating creating of index based on samples from repo linked at the start of this section

Using the index

Within the project, review the `get_attribute_info` method (*in tools/get_attribute_info.py*) to see examples of how to run a search using vector-only, hybrid (vector + keyword), or keyword-only. We've found we get the best results using a hybrid search approach. There are equivalent c#/dotnet examples found here.

Prompt template

The structuring of the prompt is crucial to the success of this project. The 3 prompt templates (1. for generating rule names, 2. for generating JSON payloads for standard selection rules and 3. for generating JSON payloads for basic selections rules) can be found in the **tools/nlp_to_rpi_prompts.py**. We're getting good results with the prompts as they are, but I expect this will change and be tweaked over time. With that said, the first thought on productizing this is ensuring it's possible for system administrators to override the prompt. It's worth mentioning again how valuable this document is wrt understanding prompt engineering and the role of system messages and examples: System message design for Azure OpenAI - Microsoft Foundry

Thoughts on productizing

- Allow the ability to override/enhance the NLP prompt outside of software release cycles
- Provide a system task that can be used for creation/maintenance/updates of cog search index
- All requests/results should be logged and users should be able to provide thumbs up/thumbs down feedback on the results so that these can be periodically reviewed and we can improve the prompting techniques

- Allow use of Azure OpenAI or non-Azure OpenAI (most SDKs I've seen support both)
- We should have caveats specifying that this is experimental and can produce unexpected results (as a lot of apps with LLM powered features/components do)
- We may want to consider putting in throttling/limits to how many NLP request a tenant can issue during a given timeframe.
- This POC uses the integration API to create object in RPI. If we add support for NLP to basic rules in the core product we may be able to come up with a simpler intermediary JSON structure for the model to output

Pricing

Azure cognitive services: Pricing - Foundry IQ | Microsoft Azure

OpenAI: Azure OpenAI Service - Pricing | Microsoft Azure